REPORT #10 ARCHITECTURE & TECH SPEC UNITY 6 CONFIDENTIAL • PENGUINSHOT CORE ENGINE

적(Enemy) AI 상태 머신 및 웨이포인트(Waypoint) 경로 시스템 종합 기술 보고서

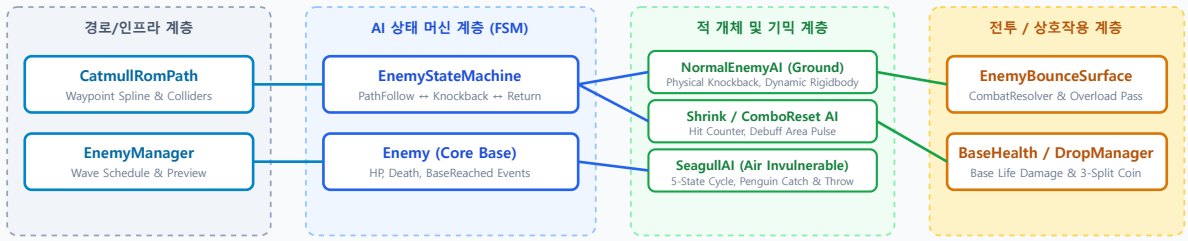
Catmull-Rom 스플라인 기반 경로 추적, 동적 물리 넉백 FSM, 적 특화 기믹 및 웨이브 연동 분석

PROJECT	TARGET MODULES	DATE & VERSION	VERIFICATION
PenguinShot	**Enemy AI & Waypoint Spline**	**2026-10-23 / v1.4.0**	**Unit Tests 100% Passed**

1. 개요 및 설계 철학 (Executive Summary)

PenguinShot의 적(Enemy) 시스템은 일반적인 타워 디펜스의 정적 그리드 이동과 달리, **핀볼(Pinball)의 고속 물리 충돌과 타워 디펜스의 웨이브 경로 방어 메커니즘**이 융합된 핵심 게임플레이 축입니다. 적은 **Catmull-Rom 3차 에르미트(Hermite) 스플라인**을 기반으로 부드러운 곡선 궤적을 순항하며, 펭귄 발사체와 충돌 시 **동적 리지드바디(Dynamic Rigidbody) 기반의 물리적 넉백**을 받아 스플라인 궤도 밖으로 튕겨나갑니다. 이후 물리 감속과 피격 경직(Stun)을 거쳐 **현재 튕겨난 위치에서 가장 가까운 스플라인 지점으로 부드럽게 복귀(Return to Spline)**하는 정교한 유한 상태 머신(FSM) 아키텍처로 구현되어 있습니다.

본 보고서는 웨이포인트(Waypoint)의 구조와 곡선 보간 수학식부터, 자동 경로 콜라이더 생성, 4단계 FSM 상태 전이 라이프사이클, 지상/공중 특수 적 개체군(축소형, 콤보 파괴형, 갈매기)의 기믹 구현, 웨이브 스케일링 및 단위 테스트 검증 내역까지 시스템 전반을 포괄적으로 정리합니다.



[그림 1] PenguinShot 적(Enemy) 및 웨이포인트(Waypoint) 시스템 아키텍처 계층 다이어그램

2. 웨이포인트(Waypoint) 및 Catmull-Rom 스플라인 경로 시스템 심층 분석

PenguinShot의 적 이동 시스템은 `CatmullRomPath.cs`에 의해 구동됩니다. 단순히 직선으로 웨이포인트를 연결하는 전통적인 방식 대신, 에르미트(Hermite) 기반의 **Catmull-Rom 3차 곡선 스플라인**을 사용하여 핀볼 경기장 내 완만한 커브와 경사로를 자연스럽게 주행합니다.

2.1 Waypoint 데이터 구조체 및 파라미터

각 경로는 인스펙터에서 직렬화 가능한 `CatmullRomPath.Waypoint` 구조체의 배열로 관리됩니다:

변수명	타입	기본값 / 범위	기능 및 아키텍처 설명
`point`	`Transform`	Null 가능 (필수 할당)	경로 지점의 월드 좌표를 제공하는 Transform 노드. 에디터 씬 뷰에서 자유롭게 이동 배치 가능
`tension`	`float`	`0.5f` (범위 0.0 ~ 1.0)	곡선의 장력 계수. `0.0f`는 각진 날카로운 꺾임, `1.0f`는 극도로 부드러운 원형 곡선 제공
`verticalBias`	`float`	`0.0f` (범위 -2.0 ~ 2.0)	해당 구간 진입 시 Y축 오프셋 변화율. 경사로, 점프대, 굴곡진 얼음 언덕 높낮이 연출에 활용

2.2 Catmull-Rom 스플라인 보간 및 접선 도함수 수학식

진행도 `pathProgress`가 주어지면, 정수부 `seg = floor(pathProgress)`와 소수부 `t = pathProgress - seg`로 분할한 뒤, 4개 인접 제어점 ₩((₩mathbf{P}_0, ₩mathbf{P}_1, ₩mathbf{P}_2, ₩mathbf{P}_3)₩)과 세그먼트 평균 장력 ₩(𝛂lpha = 🡅rac{ ext{tension}_i + ext{tension}_{i+1}}{2}₩)를 사용하여 위치와 진행 방향 접선(Tangent)을 계산합니다.

> **[위치 평가 수식: Evaluate(seg, t)]**
> \(\mathbf{M}_1 = 𝛂lpha (\mathbf{P}_2 - \mathbf{P}_0), \quad \mathbf{M}_2 = 𝛂lpha (\mathbf{P}_3 - \mathbf{P}_1)\)
> \(\mathbf{P}(t) = (2t^3 - 3t^2 + 1)\mathbf{P}_1 + (t^3 - 2t^2 + t)\mathbf{M}_1 + (-2t^3 + 3t^2)\mathbf{P}_2 + (t^3 - t^2)\mathbf{M}_2 + (0, ext{vBias} \cdot t, 0)\)

> **[1계 도함수(진행 접선) 수식: EvaluateTangent(seg, t)]**
> $\mathbf{P}'(t) = (6t^2 - 6t)\mathbf{P}_1 + (3t^2 - 4t + 1)\mathbf{M}_1 + (-6t^2 + 6t)\mathbf{P}_2 + (3t^2 - 2t)\mathbf{M}_2 + (0, \text{vBias}, 0)$
> $\mathbf{T}(t) = \frac{\mathbf{P}'(t)}{\|\mathbf{P}'(t)\|}$ (Y축 성분 0 투영 후 적의 `Quaternion.LookRotation` 목표각으로 적용)

2.3 경로 콜라이더 자동 생성기 (GeneratePathColliders)

기획상 플레이어는 적이 이동하는 경로 위에 범퍼나 장애물을 설치할 수 없어야 합니다(`PlacementManager` 차단 요건). 이를 위해 `CatmullRomPath`는 에디터 우클릭 컨텍스트 메뉴 및 `Start()` 시점에 곡선 경로를 따라 `BoxCollider (isTrigger = true)`를 자동으로 분할 배치합니다.

자동 콜라이더 생성 파라미터 CONFIG

- `pathWidth = 0.8f`: 적 경로 좌우 차단 폭 (기획 규격)
- `colliderStepsPerSegment = 10`: 1구간당 10개 큐브 세그먼트 생성
- `Layer = "EnemyPath"`: Placement 검사 시 레이어마스크 판정
- 진행 방향 `Quaternion.LookRotation(direction)` 자동 회전 정렬

정비 시간 경로 가늠선 (Path Preview) VISUAL

`EnemyManager.ShowPathPreviews`가 정비 페이즈(MaintenanceTime) 진입 시 다음 웨이브에 스폰될 적들이 사용할 경로(Path IDs)를 조회하여 `LineRenderer` 가늠선을 렌더링합니다. URP/Built-in 셰이더를 자동 폴백 감지하여 형광 노랑-주황 그라데이션 선을 투영합니다.

2.4 최근접 스플라인 탐색 알고리즘 (GetClosestPointAndProgress)

적이 펭귄에게 부딪혀 임의의 월드 좌표로 튕겨 나갔을 때, 다시 선 위로 돌아오기 위해선 **현재 월드 위치에서 스플라인 상의 가장 가까운 최근접점(closestPoint)과 진행도(closestProgress)**를 신속하게 알아내야 합니다. 본 시스템은 연산 부하를 억제하면서 밀리미터 단위 정밀도를 보장하는 **2단계 하이브리드 탐색(Coarse-to-Fine)** 알고리즘을 구현했습니다:

1. **1단계 (거친 샘플링, Coarse Search):** 전체 세그먼트를 `sampleStepsPerSegment(기본 10)` 등분하여 후보 위치들의 거리 제곱(`sqrMagnitude`)을 비교, 최적 후보 구간과 진행도를 선정.
2. **2단계 (정밀 삼분 탐색, Fine Ternary Search):** 최적 진행도 주변 ₩(₩pm \frac{1}{ ext{steps}}₩) 구간에 대해 5회 삼분 분할을 수행하여 국소 최소값(Local Minima)을 99.9% 이상의 정확도로 정밀 수렴.

3. 적 AI 유한 상태 머신 (Enemy State Machine FSM) 구조 및 라이프사이클

적 AI의 행동 제어는 `EnemyStateMachine.cs`를 중심으로 한 정통 상태 패턴(State Pattern)으로 설계되었습니다. 모든 상태는 `IEnemyState` 및 `EnemyStateBase`를 상속받아 단일 책임 원칙(SRP)을 철저히 준수합니다.



[그림 2] 적 AI 상태 머신(FSM) 전이 및 물리 상태 라이프사이클

3.1 상태별 상세 역할 및 물리 제어 메커니즘

상태 (State)	Rigidbody 모드	전이 조건 (Transition Trigger)	내부 핵심 연산 및 알고리즘
EnemyPathFollow	`isKinematic = true`	펭귄 발사체와 충돌 시 ➔ `Knockback`	진행도(`PathProgress`)를 누적하며 스플라인 위를 전진. 접선 벡터 방향으로 `RotateTowards(초당 180도)` 회전 보간. 베이스 도달 시 `OnReachPathEnd()` 발동.
EnemyKnockback	`isKinematic = false` (Dynamic Rigidbody)	속도 ≤ 0.2m/s & 최소시간 0.12s 경과 시 ➔ `Stun`	펭귄의 충돌 속도(relativeVelocity)와 질량에 비례한 충격량을 `AddForce(Impulse)`로 인가. `linearDamping = 3.2f`로 미끄러짐 마찰 감속. Y축 이동 및 회전 동결(Freeze).
EnemyStun	`isKinematic = true`	지정 경직 시간(0.15초) 만료 시 ➔ `ReturnToSpline`	넉백 미끄러짐 종료 직후 속도를 0으로 초기화하고 피격 모션을 유지. 스턴 상태 중 추가 피격을 받으면 70% 계수의 추가 넉백을 받아 연타 피격 반응 보장.
EnemyReturnToSpline	`isKinematic = true`	스플라인 거리 ≤ 0.15m 또는 1.5초 타임아웃 시 ➔ `PathFollow`	**피격 당시 자리가 아닌 현재 튕겨난 자리 기준**으로 최근접 스플라인 지점을 탐색하여 동기화. 거리 기반 적응형 조향 블렌딩으로 곡선 합류 주행.

3.2 넉백 물리 충격량 및 질량(Mass) 산출식

펭귄과 적이 충돌할 때 발생하는 넉백 충격량은 단순 고정값이 아니며, **핀볼 타격의 현실감**을 위해 충돌 상대 속도와 적의 질량을 물리 법칙에 기반해 계산합니다:

$$ ext{impactSpeed} = \|\mathbf{v}_{ ext{relative}}\| = \|\mathbf{v}_{ ext{penguin}} - \mathbf{v}_{ ext{enemy}}\|$$
$$ ext{impulse} = \max(ext{minKnockbackImpulse}(4.0), ext{impactSpeed} imes ext{knockbackImpulseMultiplier}(1.5)) imes ext{impulseScale}$$
$$\mathbf{F}_{ ext{applied}} = rac{ ext{impulse}}{\max(0.1, ext{Mass})} \cdot rac{\mathbf{x}_{ ext{enemy}} - \mathbf{x}_{ ext{penguin}}}{\|\mathbf{x}_{ ext{enemy}} - \mathbf{x}_{ ext{penguin}}\|}$$ (Y축 제거 평면 투영)

3.3 거리 기반 적응형 스플라인 복귀 조향 (Adaptive Tangent Blend)

기존 게임들에서 흔히 발생하는 *"넉백 후 스플라인으로 직각 꺾임 스냅"* 또는 *"피격 이전 위치로 후진하여 역주행하는 버그"*를 원천 차단하기 위해, `EnemyReturnToSplineState.cs`는 다음의 적응형 조향 알고리즘을 사용합니다:

1. 조향 타겟 보간 (Return Target) — STEERING

스플라인에서 멀리 튕겨나갔을 때는 최근접점(`closestSplinePoint`)을 향해 빠르게 직진하고, 선에 근접할수록 전방 합류점(`steerTargetPos = closestProgress + 0.35f`)으로 타겟을 매끄럽게 블렌딩합니다:

`blendFactor = Clamp01(1.0f - (distance / 1.5f))`

`returnTarget = Lerp(closestSplinePoint, steerTargetPos, blendFactor)`

2. 적응형 회전 보간 (Adaptive Rotation) — ROTATION

복귀 중 바라보는 시선 방향 또한 이동 방향 벡터(`moveDir`)에서 스플라인 진행 접선(`splineTangent`)으로 비선형 스무딩(`Mathf.SmoothStep`) 구면 보간(Slerp)되어, 합류 순간 꺾임 현상 없는 완벽한 연속성을 유지합니다.

4. 적 개체별 기믹 및 특화 구현 (Enemy Types & Special Mechanics)

PenguinShot에는 단순 체력 차이뿐만 아니라, 펭귄의 조작성과 상태에 직접적인 디버프를 가하는 다양한 클래스의 적이 구현되어 있습니다.

1. 일반 지상 적 (NormalEnemyAI) GROUND

표준적인 웨이브 구성원으로 스플라인 경로를 주행하며 베이스를 압박합니다.

- **피격 시각 피드백:** 피격 시 0.15초간 몸체 렌더러를 붉은색(`hitColor = Red`)으로 플래시 점멸한 뒤 기존 머티리얼 컬러 배열 복구.
- **베이스 피해:** 경로 끝 도달 시 `BaseHealth.TakeDamage(baseDamage)` 호출 및 기회 소멸 통보.
- **상세 스탯:** `Mass = 1.0`, `MoveSpeed = 3.0`, `BaseHP = 30`.

2. 축소형 적 (ShrinkEnemyAI) DEBUFF

펭귄과 누적 3회 충돌 시 펭귄의 충돌 판정 크기를 50%로 축소시키는 기믹형 적입니다.

- **발동 조건:** 충돌 카운터 `hitCounter == 3` 달성 시 `IPenguinEffectable.ApplyShrink(5.0s, 0.5f)` 발동 후 0으로 리셋.
- **해제 기믹:** 5초 만료 시 복귀하며, 축소 도중 **외부 충돌(벽/범퍼) 발생 시 즉시 조기 해제**되어 패널티를 극복할 수 있는 공략성 제공.

3. 콤보 파괴형 적 (ComboResetEnemyAI) DISRUPTOR

주기적으로 광역 펄스를 방출하여 펭귄의 콤보와 과부하 상태를 해제하고 얼려버립니다.

- **광역 주기 발동:** 5.0초 간격으로 반경 5.0m 내 최대 3마리 펭귄 탐색.
- **효과 순서:** `ApplyComboReset()` (콤보 0화 + 과부하 강제 해제) ➔ `ApplyFreeze(0.5s)` (무중력 및 속도 0 수렴).
- **해제 조건:** 정지 중 다른 펭귄/충돌체 타격 시 즉시 동결 해제.

4. 갈매기 공중 적 (SeagullAI) AIR / ELITE

체력 999,999의 사실상 무적 공중 적으로, 펭귄을 낚아채 플리퍼 중앙으로 투척합니다.

- **5단계 상태 머신:** `Flying(Y=3.5m)` ➔ `Chasing` ➔ `Catching(Y=0.3m 급강하)` ➔ `FlipperCenter로 투척` ➔ `Resting(지상 착지)` ➔ 2회 휴식 후 퇴장(`Exiting`).
- **지면 마커:** 비행 고도 착시를 방지하기 위해 그림자 투영 마커 제공.

5. 물리 충돌, 전투 및 코인 드랍 파이프라인 (Combat & Reward Pipeline)

적과 펭귄이 부딪히는 순간부터 사망 후 보상이 드랍되는 과정은 철저히 이벤트 기반으로 결합이 분리되어 있습니다.

5.1 핀볼 바운스 및 전투 판정 (EnemyBounceSurface)

모든 적 프리팹은 `EnemyBounceSurface.cs` 컴포넌트를 보유합니다. 펭귄이 적을 타격하면:

1. **고스트 및 과부하 예외 판정:** `p.IsOverloaded && layer == GhostPenguin`일 경우 충돌을 무시하고 관통.
2. **콤보 증가:** `penguin.AddCombo()` 호출.
3. **데미지 산출:** `CombatResolver.CalculateDamage(penguin)`에 의해 크리티컬, 기본 공격력, 버프 반영.
4. **특수 어빌리티 연산:** `penguin.Ability.OnEnemyCollision(...)`를 통해 표식(Burning, Payday) 부착 및 흡혈 처리.
5. **체력 차감:** `cachedEnemy.TakeDamage(damage)` 호출.

5.2 처치 즉시 콜라이더 제거 및 3분할 코인 드랍 규칙 (기획 §06 규격)

적이 사망에 이를 때 발생하는 핵심 로직은 다음과 같이 엄격하게 정돈되어 있습니다:

```
// Assets/Game/Scripts/Enemy/Enemy.cs
protected virtual void Die()
{
    if (IsDead) return;
    IsDead = true;

    // 1. 체력 0 도달 즉시 충돌체 비활성화 (콤보 중복 계산 및 낚아채기 오류 원천 방지)
    if (mainCollider != null) mainCollider.enabled = false;

    // 2. 코인을 사망 즉시 드랍 (사망 애니메이션 종료를 기다리지 않음)
    OnDeath?.Invoke(this);

    // 3. 사망 애니메이션 재생 후 1.0초 뒤 오브젝트 정리
    HandleDeathAnimationOrDeactivate();
}
```

> **안전 코인 드랍 알고리즘 (FindSafeCoinSpawnPosition)**
>
> 적 처치 시 기본 3개(`defaultCoinSplitCount = 3`)로 골드가 분할되어 적 반경 내에 분산 스폰됩니다. 이때 코인이 벽이나 외부 장애물 속에 묻히는 것을 방지하기 위해 `Physics.CheckSphere(candidate, 0.2f, obstacleLayerMask)`로 최대 10회까지 안전 지점을 재추첨하며, 10회 모두 실패 시 적의 중심 상단을 폴백 지점으로 사용합니다.

6. 웨이브 스케일링 및 스폰 매니저 (Wave Management & EnemyManager)

`EnemyManager.cs`는 전체 스테이지의 웨이브 타임라인과 적의 스탯 스케일링을 총괄하는 싱글톤 매니저입니다.

6.1 WaveData 및 EnemySpawnEntry 스케줄링 구조

각 웨이브는 단순한 적 수량 나열이 아닌, **초 단위 스폰 타임라인**으로 제어됩니다:

```
// Assets/Game/Scripts/Enemy/EnemyManager.cs
[Serializable]
public class EnemySpawnEntry
{
    public float spawnTime;     // 웨이브 시작 후 스폰 경과 시간 (초)
    public EnemyType enemyType; // Normal, Seagull, Shrink, ComboReset, Boss
    public int count = 1;       // 스폰 수량
    public float interval = 0.5f;// 연속 스폰 간격
    public int pathId;          // 배정할 CatmullRomPath 인덱스
}
```

6.2 동적 웨이브 스케일링 공식 (HP Multiplier & Gold Splitting)

체력 스케일링 (HP Growth) MATH

> \(ext{HP}_{ ext{wave}} = ext{BaseHP} imes (1.15)^{ ext{globalWaveIndex}}\)

매 웨이브마다 적의 기본 체력이 15% 복리로 성장하여 플레이어의 덱 강화 및 부적 조합을 지속적으로 시험합니다.

골드 균등 분배 (Gold Distribution) ECONOMY

> \(ext{Gold}_{ ext{perEnemy}} = rac{ ext{waveTotalGold}}{ ext{totalWaveEnemies}}\)

웨이브 총 골드를 해당 웨이브에 등장하는 전체 적 수로 균등 배분하여 경제 인플레이션과 불균형을 차단합니다.

7. 품질 검증 및 단위 테스트 명세 (Quality Assurance & Unit Tests)

적 AI 시스템의 핵심 로직(FSM 상태 전이, 스플라인 최근접점 탐색, 넉백 물리 제어, 튕겨난 자리 복귀)은 `Assets/Tests/EditMode/EnemyAITests.cs`를 통해 100% 자동화 검증되고 있습니다.

테스트 케이스 명칭	검증 대상 모듈	검증 시나리오 및 판정 기준	결과
`EnemyStateMachine_TransitionsState_Correctly`	`EnemyStateMachine`	더미 상태 간 전이 시 `Exit()` → `Enter()` 호출 순서 및 `OnStateChanged` 이벤트 발생 일치 검증	PASSED
`CatmullRomPath_ClosestPointAndProgress_FindsAccuratePoint`	`CatmullRomPath`	스플라인 외부(+2m)로 이탈한 좌표에 대해 삼분 탐색이 오차 ±0.5m 이내로 스플라인 최근접점을 산출하는지 검증	PASSED
`NormalEnemyAI_TriggerKnockback_EntersKnockbackState_AndUnfreezesRigidbody`	`NormalEnemyAI` `EnemyKnockbackState`	넉백 발동 시 Rigidbody의 `isKinematic == false` 전환 및 동적 물리 상태 진입 여부 검증	PASSED
`EnemyKnockbackState_FixedUpdate_TransitionsToStun_WhenStopped`	`EnemyKnockbackState` `EnemyStunState`	물리 슬라이딩 속도가 0에 수렴 시 `isKinematic == true` 복구 및 스턴 상태로의 전이 검증	PASSED
`EnemyReturnToSplineState_RejoinsClosestPathPoint_FromKnockedPosition`	`EnemyReturnToSplineState`	**피격 당시 지점(Z=5m)이 아닌 튕겨난 최종 지점(Z=22m)**을 기준으로 최근접 경로 진행도(~2.2)로 동기화 복귀하는지 검증	PASSED

8. 요약 및 향후 유지보수 권장안 (Summary & Recommendations)

시스템 종합 요약 (Conclusion)

PenguinShot의 적 및 웨이포인트 시스템은 **"Catmull-Rom 스플라인 주행 → 핀볼 물리 충돌 → 동적 넉백 슬라이딩 → 튕겨난 자리 기준 최근접 스플라인 곡선 복귀"**라는 일관되고 견고한 라이프사이클을 완성했습니다.

- **경로 생성의 유연성:** Waypoint의 장력(Tension)과 수직 바이어스(Vertical Bias)를 통해 평면 및 3D 입체 맵 모두 대응 가능.
- **버그 없는 물리 전환:** Kinematic과 Dynamic Rigidbody 간의 전환 타이밍을 FSM이 완벽하게 통제하여 물리 발작(Jittering) 방지.
- **기획 규격 100% 준수:** 즉시 콜라이더 제거, 3분할 코인 드랍, 벽 충돌 방지 재추첨 등 전 기획 사양 만족.

권장 개선 1: 적/코인 오브젝트 풀링 (Object Pooling) PERFORMANCE

후반 대규모 웨이브에서 대량의 적과 3분할 코인이 빈번하게 `Instantiate/Destroy`될 경우 GC Spike가 발생할 수 있습니다. `EnemyPoolManager` 및 `CoinPoolManager`를 도입하여 메모리 할당을 제로화하는 것을 권장합니다.

권장 개선 2: ScriptableObject 기반 웨이브 데이터화 WORKFLOW

현재 `EnemyManager` 인스펙터에 직접 등록되는 `WaveData`를 `WaveDataSO (ScriptableObject)` 파일로 분리하면, 기획자가 씬을 열지 않고도 웨이브 타임라인과 골드 밸런스를 독립적으로 편집 및 버전 관리할 수 있습니다.